BLOCKCHAIN

Smart Contracts Visual Guide

EVM execution, the gas model, Solidity basics, and the vulnerabilities auditors hunt.

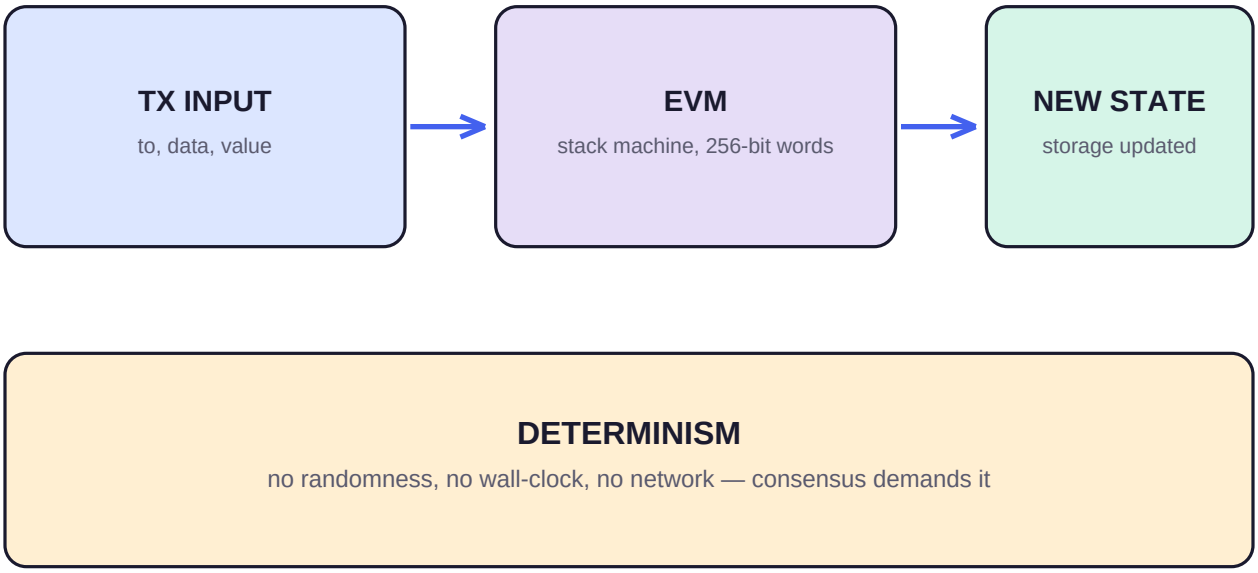
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

EVM Execution Model

A deterministic world computer: same input + same state = same output, everywhere.



Every node replays every transaction. The EVM is slow by design — verifiability over speed.

STEP 2/6

The Gas Model

Every opcode has a price. You prepay; unused gas refunds.

WHY GAS EXISTS

halts the halting problem: infinite loops just run out of money

Storage writes (SSTORE) are the most expensive ops — minimize on-chain state.

STEP 3/6

Solidity Basics: Contract Anatomy

State variables, functions, and who may call them.

contract Vault { uint balance; ... }

state lives on-chain forever

external/public	callable from outside / anywhere
internal/private	only this contract / + derived
view / pure	reads state / touches nothing — free to call off-chain

STEP 4/6

Storage vs Memory vs Calldata

Three data locations, three cost profiles.

STORAGE

persistent, on-chain — $$$$ (SSTORE ~20k gas)

MEMORY

per-call scratch space — cheap, wiped after

CALLDATA

read-only function inputs — cheapest

Audit rule of thumb: every storage write should justify its gas. Pack variables into 32-byte slots.

STEP 5/6

Common Vulnerabilities

The hits auditors find again and again.

Reentrancy	external call before state update -> drained (checks-effects-interactions!)
Access control	missing onlyOwner -> anyone calls admin functions
Oracle manipulation	single-source price feeds get gamed — use TWAP/decentralized oracles
Integer issues	pre-0.8 overflow/underflow; still watch rounding in DeFi math
Front-running	public mempool reveals intent — commit-reveal or private relays

STEP 6/6

Audits & Secure Patterns

How serious teams ship contracts that hold money.

Checks-Effects-Interactions	update state BEFORE calling out — kills reentrancy
Battle-tested libs	OpenZeppelin for tokens, access control, upgrades
Tests + fuzzing	unit, invariant, and fuzz tests — Foundry/Hardhat
External audit	independent review, then bug bounty before real TVL

STEP ✓/✓

Key Takeaways

Smart Contracts — Remember

- The EVM is a deterministic stack machine: same input and state, same result on every node.
- Gas prices every opcode; you prepay, and infinite loops just run out of money.
- Solidity: state variables persist; function visibility and mutability define the attack surface.
- Storage is persistent and expensive; memory is per-call; calldata is cheapest.
- Top vulnerabilities: reentrancy, access control, oracle manipulation, rounding, front-running.
- Ship with checks-effects-interactions, audited libraries, fuzzing, external audit, and a bounty.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev